



Sharif University of Technology

Department of Electrical Engineering

Course:

ASIC/FPGA

Report:

ASIC/FPGA Project - Group 8

Weekly Report II

Submitted by:

Sina Hashemi

402102668

Erik Hartouni

401102778

Ali Behrad

401101321

Mohammadhossein Sabzalian

402101906

Instructor:

Dr. Baharvand

Department of Electrical
Engineering

Term: Spring 2026

Project Management and Verification Hand-off

Week 2 followed the split established in Week 1: **Erik Hartouni** and **Ali Behrad** owned the verification track, delivering the self-checking testbench `tb_otp_controller.v` together with the supporting simulation ROM model and OTP memory-map header. **Mohammadhossein Sabzalian** and **Sina Hashemi** remained responsible for the RTL, and in practice this meant reacting to the testbench as it came online: the first RTL drop compiled cleanly and matched the FSM described in Week 1, but did not yet behave correctly once real address/data timing was exercised end-to-end. Bringing `otp_controller.v` to a passing state against the new testbench therefore required two rounds of RTL correction, detailed in Section , before verification could be closed out.

Others:

- **Repository update:**

The testbench, simulation ROM model (`otp_sim_rom.v`), memory-map header (`otp_map.vh`), and a golden OTP image (`otp_image.mem`) were committed alongside the RTL sources and synchronized to the server, matching the Week 1 folder structure.

Week 2 Plan vs. Actual

Planned Activity (Weekly Schedule)	Responsible	Status
Complete testbench (drivers, monitors)	C, D	Done
Run basic sanity tests (nominal case)	C, D	Done
Functional verification (all test cases from DRS)	A, B, C, D	Done
Debug RTL issues found during verification	A, B	Done (2 issues, Sec.)
Integration Lead (Week 2) reports progress	B	Done

Table 1: Week 2 deliverables against the schedule defined in the project plan.

Testbench Architecture and Verification Methodology

The testbench implements the procedure of DRS Section 7 as a single self-checking module built from three layers: software reference-model **helper functions**, reusable **helper tasks** that drive the DUT, and a lightweight pass/fail **scoreboard**.

Helper Functions: Software Reference Models

Rather than trusting the RTL to check itself, the testbench re-implements the OTP encoding rules in behavioral Verilog so that expected values are computed independently of the design under test.

- **crc_step()** / **sw_crc16()**: a bit-serial software model of the CRC-16/CCITT taps, identical to `crc16_ccitt.v`, used to compute the expected checksum over the first 183 bits of Bank0 for any synthetic image.
- **getfield()**: extracts an arbitrary bit range from a 256-bit image, used to build the expected value of every output register directly from the golden OTP image rather than from a second hard-coded copy of the memory map.
- **build_patch()** / **build_image()**: pack a 28-bit patch record (`valid`, `id`, `data`, `tag`) and a full 256-bit OTP image respectively, with an option to deliberately corrupt the stored CRC for the CRC-error test.

Listing 1: Software CRC-16/CCITT Model and Field Extraction Helper (Testbench)

```
1 function automatic [15:0] crc_step(input [15:0] crc, input bit_in);
2     reg xor_bit;
3     begin
4         xor_bit      = bit_in ^ crc[15];
5         crc_step[15] = crc[14] ^ xor_bit;
6         crc_step[12] = crc[11] ^ xor_bit;
7         crc_step[5]  = crc[4]  ^ xor_bit;
8         crc_step[0]  = xor_bit;
9         // remaining bits are direct pass-throughs of crc[]
10    end
11 endfunction
12
13 function automatic [19:0] getfield(input [255:0] img, input integer
14     lo, input integer hi);
15     integer i;
16     reg [19:0] v;
17     begin
18         v = 20'd0;
19         for (i = 0; i <= hi - lo; i = i + 1)
20             v[i] = img[lo + i];
21         getfield = v;
22     end
23 endfunction
```

- **Independent Golden Model:** `sw_crc16` mirrors `crc16_ccitt.v` tap-for-tap, so a passing CRC check confirms the RTL polynomial implementation rather than merely confirming self-consistency.

- **Map-Driven Expectations:** `getfield` is called with the same ‘OTP*_LO/HI’ bounds as the RTL (via `otp_map.vh`), so Test 2 expectations track the memory map automatically if it is ever revised.

Helper Tasks: Driving the DUT

Listing 2: Reset, Watchdog, and Reprogram-and-Run Tasks (Testbench)

```
1 task do_reset;
2     begin
3         rst_n = 1'b0;
4         repeat (4) @(posedge clk);
5             @(negedge clk);
6         rst_n = 1'b1;
7     end
8 endtask
9
10 task wait_for_done;
11     integer timeout;
12     begin
13         timeout = 0;
14         while (!done && timeout < 1000) begin
15             @(posedge clk);
16             timeout = timeout + 1;
17         end
18         check(done, "watchdog: done asserted before timeout");
19     end
20 endtask
21
22 task load_and_run(input [255:0] image);
23     begin
24         rst_n = 1'b0;
25         @(negedge clk);
26         u_rom.otp_rom = image; // poke simulated OTP storage directly
27         do_reset;
28         wait_for_done;
29     end
30 endtask
```

- **Deterministic Reset Pulse:** `do_reset` holds `rst_n` low for four full clock cycles and releases it on a `negedge`, avoiding a race between reset release and the next clock edge sampled by the DUT.
- **Hang Protection:** `wait_for_done` bounds every test at 1000 cycles ($\approx 3\times$ the ~ 320 -cycle nominal load latency from DRS §3.3), turning a stuck FSM into a reported failure instead of a simulation timeout.
- **Re-programming a Fresh Part:** `load_and_run` writes directly into the simulated ROM’s storage array before each reset. Since real OTP is write-once, this models loading a *new, unprogrammed* simulated die for every synthetic test case rather than reprogramming the DUT itself.

Scoreboard

The `check()` task is the single point through which every assertion in the testbench is reported: it prints a [PASS]/[FAIL] line with a descriptive message and increments `pass_cnt` or `fail_cnt` accordingly, so the run ends with one unambiguous summary line rather than requiring the log to be read end-to-end.

Test Plan and Results

Six directed tests were implemented, covering DRS §7.1 items #1–#5 in full and the optional CRC-error check (#6). All tests ran against the 131.072 kHz clock specified in DRS §2.3 (CLK_PERIOD_NS = 7629.39).

Test	Purpose	Key Checks	Result
1	Reset behaviour and full read sweep	Quiet during reset, busy/done timing, all 256 addresses read, no re-read after done	10/10 PASS
2	Correct field extraction	All 13 output registers vs. golden image (otp_image.mem)	13/13 PASS
3	Patch override	Valid Patch0 overrides RO_TRIM_CODE; other fields and CRC unaffected	4/4 PASS
4	Multiple patches	Patch1 takes precedence over Patch0 on the same field	2/2 PASS
5	Invalid patch rejection	PATCH_VALID=0 and wrong PATCH_TAG are both ignored	2/2 PASS
6	CRC error detection (optional, DRS §6.5)	crc_error low for a matching image (6a), high for a corrupted image (6b)	4/4 PASS

Table 2: Directed test summary, per DRS Section 7.1.

Test 6: CRC Error Detection

Using `build_image()`, two synthetic images were constructed to exercise the CRC comparison path directly against `sw_crc16()`, the same independent software model already used for Test 2.

- **Test 6a — Good CRC:** All fields are packed into bits [182:0] of the image, `sw_crc16()` computes the expected CRC-16 over that same range, and – since `good_crc=1` – the *correct* computed value is written into bits [198:183], so the stored and computed checksums agree by construction.
- **Test 6b — Bad CRC:** Fields are packed into bits [182:0] and `sw_crc16()` computes the expected CRC-16 exactly as in 6a, but this time `good_crc=0`: the stored bits [198:183] are deliberately left mismatched against the freshly computed value, injecting the error `crc_error` is meant to catch.

Listing 3: Test 6a/6b – Image Construction (Testbench)

```
1 // Test 6a - Good CRC
2 build_image(image, ..., good_crc=1);
3 // 1. pack all fields into bits [182:0]
```

```

4 // 2. crc = sw_crc16(image, 182, 0)      -- compute CRC16 over bits [1
    82:0]
5 // 3. good_crc=1 -> image[198:183] = crc  (store CORRECT CRC)
6
7 // Test 6b - Bad CRC
8 build_image(image, ..., good_crc=0);
9 // 1. pack all fields into bits [182:0]
10 // 2. crc = sw_crc16(image, 182, 0)      -- compute CRC16 over bits [1
    82:0]
11 // 3. good_crc=0 -> image[198:183] left mismatched (CORRUPTED CRC)

```

Listing 4: Test 6 Regression Log (Simulation Console Output)

TEST 6: CRC error detection

[PASS] watchdog: done asserted before timeout
[PASS] T6a: crc_error is low when stored CRC matches computed CRC
[PASS] watchdog: done asserted before timeout
[PASS] T6b: crc_error is high when stored CRC does not match computed CRC

SUMMARY: 39 PASSED, 0 FAILED (of 39 checks)

RESULT: ALL TESTS PASSED

Simulation complete via \$finish(1) at time 17238606705 PS + 0
../rtl_sources/tb_otp_controller.v:491 \$finish;
xcelium>

Regression Summary Across the six directed tests, all **39 self-checking assertions passed** (pass_cnt = 39, fail_cnt = 0). This satisfies every Section 7.3 pass/fail criterion: reset behaviour, continuous busy during load, correct field values, correct patch precedence, a single-cycle done pulse, no spurious re-reads after completion, and correct CRC error detection for both a matching and a corrupted image.

RTL Corrections Driven by Verification

The RTL passed compilation in Week 1 but failed the first real testbench run. Two independent issues were found and fixed; both are timing-related and neither was visible from the FSM diagram or pseudocode alone, which is precisely what the testbench was built to catch.

ID	Symptom / Found By	Root Cause	Fix
BUG-01	Every accumulated field was reconstructed one address off; Test 2 field mismatches	ROM registers <code>otp_data_out</code> one cycle after <code>otp_addr/otp_read_en</code> (DRS §3.3), but accumulation logic indexed with the <i>current-cycle</i> <code>addr</code>	Added delayed <code>addr_d/ren_d</code> copies; accumulation and state-transition boundary checks now use <code>addr_d/ren_d</code>
BUG-02	<code>busy/otp_read_en</code> observed asserting while <code>rst_n</code> was still low; Test 1 reset-quiet checks	Reset landed directly in <code>STATE_READ_BANK0</code> , so the FSM could start issuing reads before reset release synchronized cleanly	Reset now lands in <code>STATE_IDLE</code> ; a one-shot <code>start_pending</code> register (armed at reset, consumed on the first exit from <code>STATE_IDLE</code>) still starts the load automatically, one cycle after release

Table 3: RTL issues found during Week 2 verification.

BUG-01: Read/Address Pipeline Skew

Listing 5: Field Accumulation – Before (indexed by same-cycle `addr`) vs. After (indexed by delayed `addr_d/ren_d`)

```
1 // --- Before (Week 1) ---
2 if (otp_read_en)
3     if (addr >= `OTP_RO_TRIM_LO && addr <= `OTP_RO_TRIM_HI)
4         tmp_ro_trim[addr - `OTP_RO_TRIM_LO] <= otp_data_out;
5
6 // --- After (Week 2 fix) ---
7 addr_d <= addr;           // re-create the address that otp_data_out
8 ren_d  <= otp_read_en;    // actually corresponds to, one cycle later
9 ...
10 if (ren_d)
11     if (addr_d >= `OTP_RO_TRIM_LO && addr_d <= `OTP_RO_TRIM_HI)
12         tmp_ro_trim[addr_d - `OTP_RO_TRIM_LO] <= otp_data_out;
```

- **Root Cause:** `otp_sim_rom.v` registers its output

(otp_data_out <= otp_rom[otp_addr]), so data for address N is only valid the cycle *after* N was presented. The original controller sampled that data against whatever address it happened to be driving that same cycle – off by one position for every single field.

- **Fix Scope:** The same addr_d/ren_d pair was also used to re-time the STATE_READ_BANK0→STATE_READ_PATCH and STATE_READ_PATCH→STATE_APPLY boundary checks, so the FSM advances only once the last bit of a region has actually been *sampled*, not merely *issued*.
- **Verification:** Confirmed by Test 2 (all 13 output registers now match the golden image bit-for-bit) and Test 3/4 (patch data lands in the correct field).

BUG-02: Reset Entry and the Autonomous Start

Listing 6: Sequential State Register – Before vs. After, and the New One-Shot Starter

```
1 // --- Before ---
2 always @(posedge clk or negedge rst_n)
3     if (!rst_n) state <= STATE_READ_BANK0;
4     else state <= next_state;
5
6 // --- After ---
7 always @(posedge clk or negedge rst_n)
8     if (!rst_n) state <= STATE_IDLE;
9     else state <= next_state;
10
11 // New: one-shot starter, armed on reset, consumed on first IDLE exit
12 always @(posedge clk or negedge rst_n)
13     if (!rst_n) start_pending <= 1'b1;
14     else if (state == STATE_IDLE && start_pending) start_pending <= 1'b0;
15
16 // STATE_IDLE now waits for the one-shot instead of leaving
17 // unconditionally
18 // next_state = (state == STATE_IDLE) ? (start_pending ?
19 // STATE_READ_BANK0 : STATE_IDLE) : ...
```

- **Root Cause:** DRS §6.3 requires busy=0, done=0, and (implicitly) a quiet otp_read_en for as long as rst_n is low. Landing directly in STATE_READ_BANK0 on reset removed the safety margin between reset assertion and the FSM's first read, which Test 1's dedicated reset-quiet checks were written specifically to catch.
- **Fix:** Reset now lands in STATE_IDLE. DRS §3.2 step 1's requirement that the read start with *no external trigger* is preserved by start_pending: it is armed the instant reset is asserted and is consumed the first time the FSM is actually in STATE_IDLE after release, so the load still begins automatically, one clock after rst_n deasserts.
- **Verification:** Confirmed by Test 1 (busy is low while rst_n is held low, no busy glitch observed during reset, busy asserts shortly after reset release).

Engineering Ambiguities Resolved

Patch Address Correction: Week 1's memory-collision resolution placed Patch0 at OTP bits 196–223, following the base DRS text literally. Implementing the memory-map header this week against the actual field boundaries showed that CRC16 occupies bits [198:183] (DRS §3.1), so a Patch0 window starting at bit 196 would overlap the CRC field by three bits. `otp_map.vh` and the RTL were corrected to place **Patch0 at bits 199–226** and **Patch1 at bits 227–254**, immediately *after* the CRC field ends, eliminating the overlap while keeping every patch record at its required 28-bit width. This also resolves an internal inconsistency in the base DRS document itself, which states the 196–223/224–251 ranges even though they conflict with its own CRC bit range.

Why This Matters Because the patch window is read *after* Bank0 in STATE_READ_PATCH and shares the same 8-bit `addr` counter as the CRC field, an off-by-three overlap would have caused Patch0's tag/valid bits to be corrupted by whatever CRC bits fell in the shared range – a failure mode that would only surface once real patch test vectors (Tests 3–5) were exercised, not from static inspection of the block diagram.